FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Wed 5 July 2017 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA: SIGNATURE:

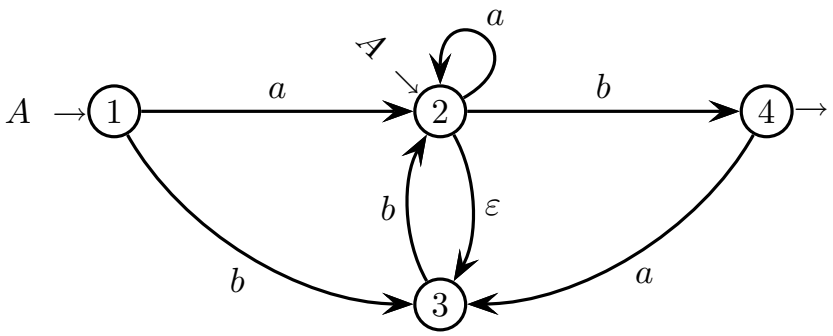
(or PERSON CODE)

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
  1. Theory (80%): Syntax and Semantics of Languages
     - regular expressions and finite automata
     - free grammars and pushdown automata
     - syntax analysis and parsing methodologies
     - language translation and semantic analysis
  2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic finite-state automaton A below, over the two-letter alphabet $\Sigma = \{\ a,\ b\ \}$, with two initial states 1 and 2, and with one final state 4:



Answer the following questions:

(a) Find the shortest string of the regular language $L(A)$ that is recognized by automaton A with at least two different computations and that therefore is ambiguous, and suitably justify your answer. Provide evidence that the string is the shortest one with such a property.

(b) Transform automaton A and obtain an automaton A', possibly still nondeterministic, that has only one initial state and does not have any spontaneous transitions (ε-transitions), by properly cutting such transitions.

(c) By using the Berry-Sethi method, obtain a deterministic automaton A'' equivalent to the original nondeterministic automaton A.

(d) (optional) Minimize the deterministic automaton A'' found at point (c). Then write a regular expression R equivalent to the automaton A'' found. Notice: for this whole question, you may proceed systematically or informally, but in the latter case provide some evidence of correctness.

---

2 Free Grammars and Pushdown Automata 20%

1. Consider the following language L over the three-letter alphabet $\Sigma = \{\ a,\ b,\ c\ \}$:

$$L = \left\{ w\ c^n \;\middle|\; \begin{array}{l} w \in \{\ a,\ b\ \}^* \ \text{ and } \ n > 0 \\ \text{and } \ (\ \#_a(w) = n \ \text{ or } \ \#_b(w) = n\ ) \end{array} \right\}$$

For instance: $b\,c,\ b\,a\,a\,c\,c,\ a\,b\,a\,a\,b\,c\,c \in L$; while $a\,a\,c,\ b\,c\,c,\ a\,b\,a\,a\,c\,c \notin L$.

Answer the following questions:

(a) Write a *BNF* grammar G (no matter if ambiguous) that generates language L.

(b) Sketch the syntax trees of the strings $b\,c,\ b\,a\,a\,c\,c \in L$.

(c) (optional) Determine if grammar G is ambiguous and justify your answer: if it is ambiguous then provide a string with two or more syntax trees, else argue that it does not generate any ambiguous string.

---

2. Consider a fragment of a programming language that is structured according to the following specifications:

   - The program consists of a declaration section followed by an execution section.
   - The declaration section is a possibly empty sequence of typed variable lists; commas "," separate the variables in a list and a semicolon ";" terminates each list in the sequence.
   - There are two variable types: scalars (i.e., integers) and fixed-size one-dimensional arrays of scalars (i.e., integers).
   - The execution section consists of a non-empty list of statements, each one terminated by a semicolon ";".
   - There are two statement types:
     - assignment: a scalar or an array element = an arithmetic expression
     - procedure call: name (possibly empty list of args separated by comma ",")
   - An arithmetic expression may consist of numbers, scalars, array elements, function calls (structured like procedure calls), operators of addition "+", subtraction "−" (possibly unary), multiplication "∗" and division "/". The operator precedences are as usual. Parenthesized subexpressions are not allowed, but an array element may be indicized by an arithmetic expression.
   - A procedure or function parameter may be a number, a scalar, an array element or an arithmetic expression.
   - Identifiers and numbers are represented by the terminals id and num, respectively.

   Further detailed information about the lexical and syntactic structure of the programming language can be inferred from the example below:

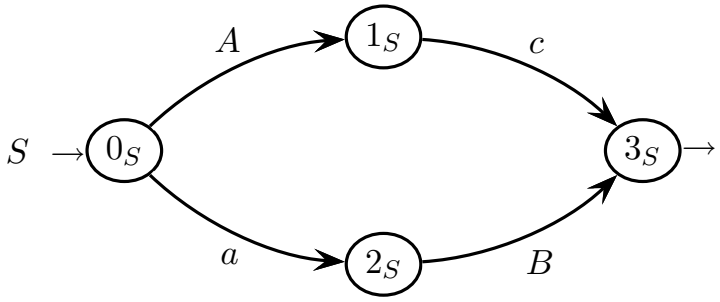
```
int a, b, c;

int v[100], d, z[50];

a = a + b * c;

v[7] = funct (a, b + 5);

proc (a, v[a + b]);

b = myfun ( );

z[c] = -d;
```

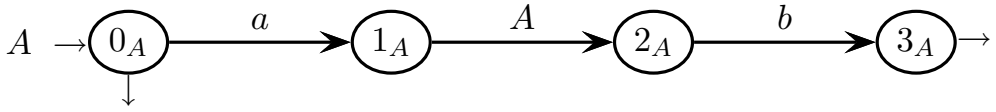
   Write a grammar, *EBNF* and unambiguous, that models the sketched language.

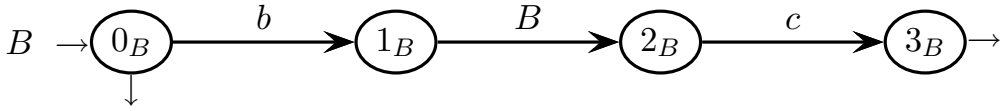
---

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G, represented as a machine net over the three-letter terminal alphabet $\Sigma = \{\ a,\ b,\ c\ \}$ and the three-letter nonterminal alphabet $V = \{\ S,\ A,\ B\ \}$ (axiom S):







Answer the following questions:

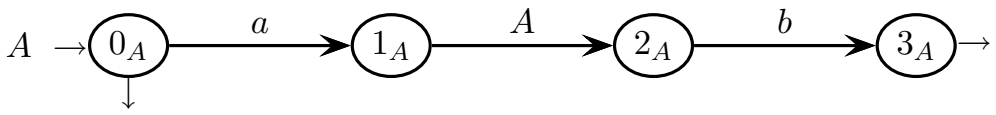
(a) Draw the complete pilot of grammar G, say if grammar G is of type $ELR(1)$, and shortly justify your answer. If grammar G is not $ELR(1)$ then highlight all the conflicts in the pilot.

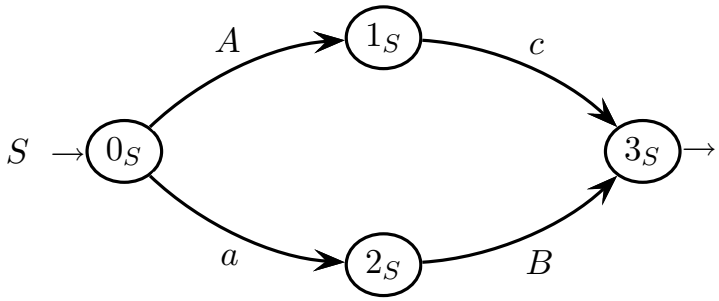
(b) Write the necessary guide sets on the arcs of the machine net, show that grammar G is not of type $ELL(1)$, based on the guide sets, and shortly justify your answer (for the guide sets please use the figure prepared on the next page).

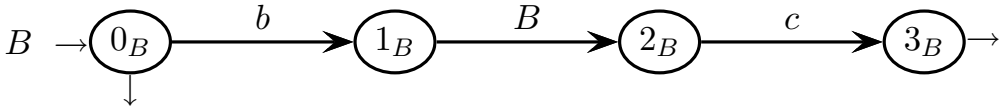
(c) (optional) Say if grammar G is of type $ELL(k)$ for some $k > 1$, and provide an adequate justification to your answer.

(d) Analyze string $a\,b\,c$ by using the Earley method, and determine if the string belongs to language $L(G)$, justifying your answer based on the contents of the Earley vector.

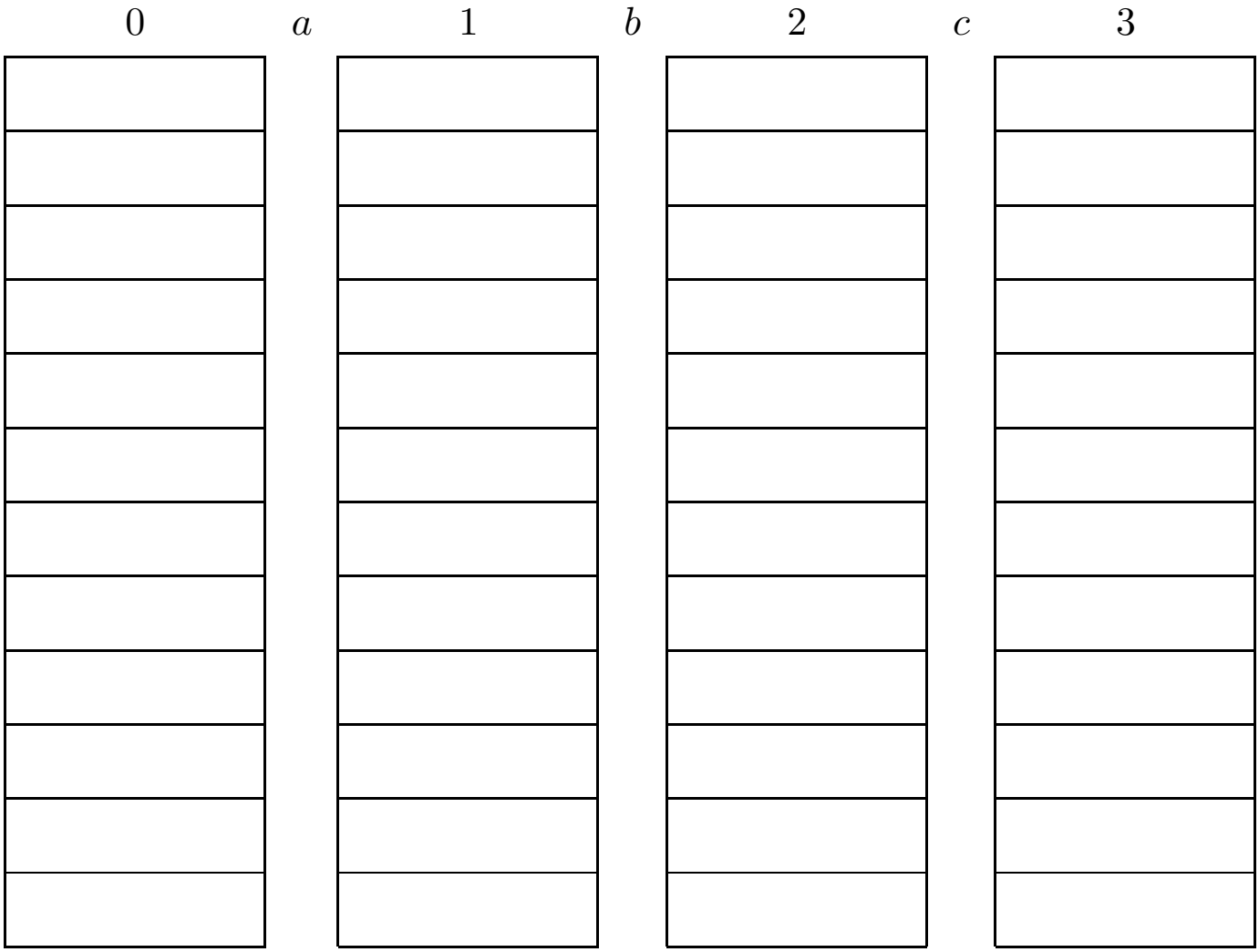
please here draw the call arcs and write the guide sets with $k = 1$







Earley vector of string $a\ b\ c$ (to be filled in)
(the number of rows is not significant)



4 Language Translation and Semantic Analysis 20%

1. The *BNF* source grammar G below generates a non-empty list of elements a and b, separated by element z (axiom S):

$$G \begin{cases} S & \rightarrow & a \mid b \\ S & \rightarrow & a\,T \mid b\,T \\ T & \rightarrow & z\,S \end{cases}$$

   Please answer the following questions:

   (a) Write a destination grammar associated to the source grammar G, without changing G, that mirrors the source string. For instance:

$$a\ z\ a\ z\ b \ \mapsto\ b\ z\ a\ z\ a$$

   To verify it, draw the source and destination syntax trees (if you wish, overlap the two trees) of the translation example.

   (b) Could the translation of point (a) be defined by a formalism simpler than a translation grammar (or scheme) ? Please adequately justify your answer.

   (c) (optional) The source grammar G is of type $ELL(1)$. For nonterminal S, write a recursive descent syntactic procedure that also outputs the translation.

---

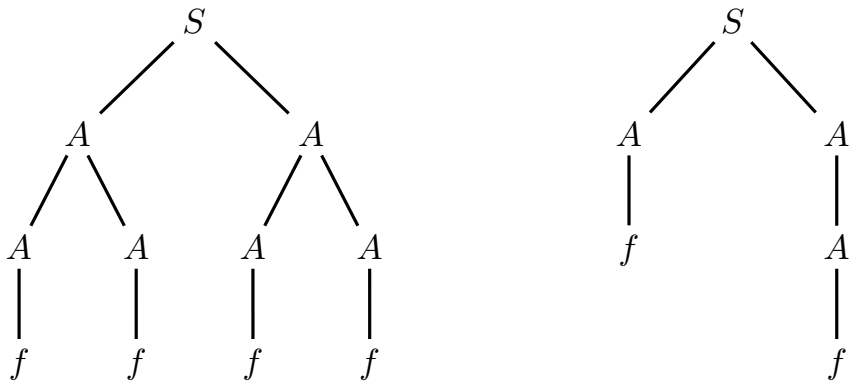
2. A binary tree is said to be *triangular* if and only if both these conditions are satisfied:

   - all its leaves are at the same *depth*, where the depth of a node is the length of the path from the root to that node
   - the number of leaves is equal to 2^{height}, where the height of the tree is the length of the path from the root to the deepest non-leaf node

   We encode binary trees through syntax trees generated by this grammar (axiom S):

$$\begin{cases} 1\colon\ S \rightarrow A\ A \\ 2\colon\ A \rightarrow A\ A \\ 3\colon\ A \rightarrow A \\ 4\colon\ A \rightarrow f \end{cases}$$

   The figure below shows a triangular tree (left) and a non-triangular tree (right).



   Notice that, according to the above definitions, the tree on the right side of the figure has one leaf at depth 2 and one leaf at depth 3, and that the height of both trees is 2.

   We intend to compute, in a boolean attribute *tri* of the root node of the syntax tree, the property of the tree being triangular. The attributes to use are already assigned, see the table on the next page. Other attributes are unnecessary and should not be added.

   Answer the following questions:

   (a) Write an attribute grammar (in the table prepared on the next page), based on the above syntax, that defines the attribute values. It is required to use only the attributes illustrated in the table on the next page.

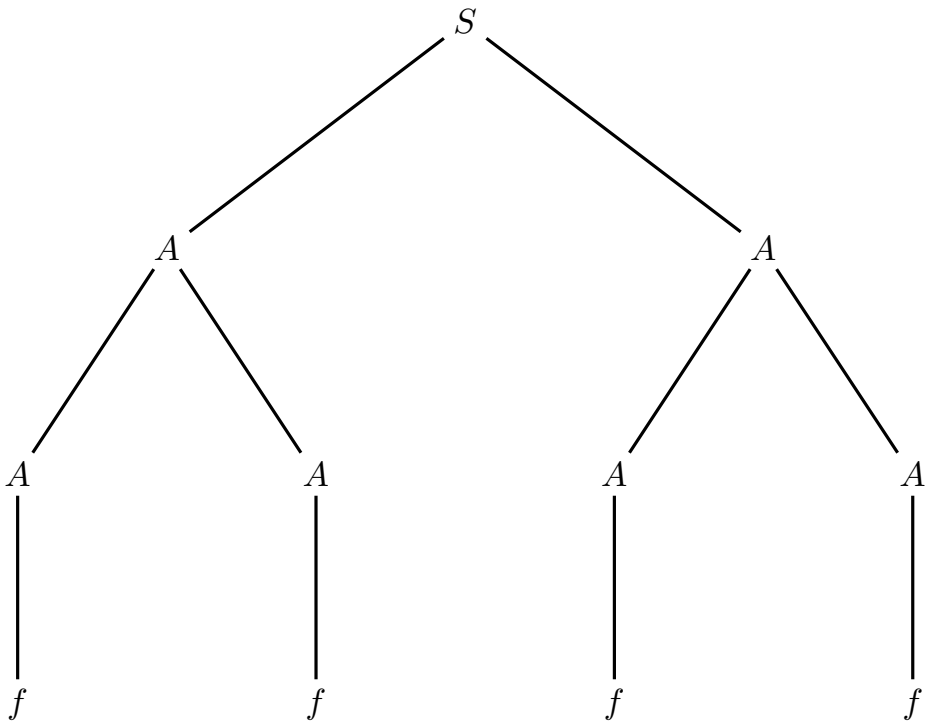
   (b) Decorate the two sample syntax trees with the appropriate attribute values. Use the two syntax trees prepared on the next page.

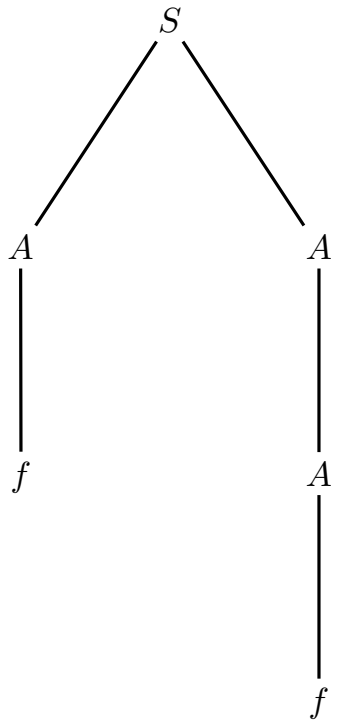
   (c) (optional) Determine if the attribute grammar is of type one-sweep and if it satisfies the L condition, and reasonably justify your answers (do not use generic or tautological sentences).

attributes assigned to be used

type	*name*	*domain*	*(non)term.*	*meaning*
right	d	integer	A	depth of the node from root
left	*nol*	integer	S, A	number of leaves of the subtree rooted at the node
left	h	integer	S, A	height of the subtree rooted at the node
left	*mind, maxd*	integer	S, A	minimal (respectively, maximal) depth of a leaf of the subtree rooted at the node; notice that for the left sample tree it holds $mind = maxd = 3$, while for the right one it holds $mind = 2$ and $maxd = 3$
left	*tri*	boolean	S	true if the tree is triangular, otherwise false

syntax trees to be decorated - question (b)





attribute grammar to write - question (a)

#	*syntax*	*semantics*
1:	$S_0 \rightarrow A_1\ A_2$	
2:	$A_0 \rightarrow A_1\ A_2$	
3:	$A_0 \rightarrow A_1$	
4:	$A_0 \rightarrow f$	